Physics-Informed Learning for Cyber Dynamics and Control

Sparse observations, parameter inference, structured mechanisms, and constrained neural control

Lu-Xing Yang

This note develops physics-informed neural networks and model-and-data learning for cyber propagation. It follows one complete inverse-SIR experiment before extending to control, PMP residuals, learned transmission mechanisms, network inference and hybrid events. Derivatives, constraints, observation masks and independent ODE replay are derived explicitly so that each implemented loss has a clear mathematical meaning. The worked examples use the same state and decision semantics as the companion control and learning notes. Prerequisites are elementary ODEs, multivariable calculus, linear algebra and basic Python/PyTorch.

How to use this note

Read the model and the mathematical derivation before changing the code. Run the first example, inspect its state and decision curves, and then use the worked extensions. The equations, assumptions and interpretation are included here; the repository supplies the executable implementations.

Contents

1 What is unknown, and what information is available? 4
1.1 Why these inference questions arise in cyber control . . . 4
1.2 Task, output and implementation . . . 4
1.3 Install and run . . . 4

2 Mechanistic models and observation maps 5
2.1 The SIR model used for parameter learning . . . 5
2.2 Measurements are not latent-state truth . . . 5
2.3 Other cyber mechanisms retain the same workflow . . . 6

3 Represent functions and constraints in a neural model 6
3.1 The state network and time scaling . . . 6
3.2 Hard initial conditions and physical state constraints . . . 6
3.3 Parameter, control and costate outputs . . . 6
3.4 Differentiate each output component . . . 7

4 Forward and inverse PINN losses 7
4.1 Write the residual before writing the optimizer . . . 7
4.2 One training iteration in the repository . . . 8
4.3 Scale losses and sample time deliberately . . . 8

5 The first experiment: fit, replay and compare 8
5.1 Use an ordinary ODE fit as a reference . . . 8
5.2 Inspect the six outputs . . . 9
5.3 Identifiability is an information question . . . 9

6 Neural optimal control 9
6.1 Distinguish a decision variable from a physical parameter . . . 9
6.2 Direct neural control . . . 10
6.3 PMP-informed control derives a different loss . . . 10
6.4 The correct residual at a control bound . . . 10

7 What accuracy and optimality can be concluded? 11
7.1 Separate the four outputs of an inverse-control study . . . 11
7.2 A residual-to-trajectory estimate . . . 11
7.3 Necessary conditions and sufficient verification . . . 12

8 PIDL: learn a structured missing mechanism 12
8.1 Conservation and positivity are separate properties . . . 12
8.2 A correction is not automatically an identified mechanism . . . 13
8.3 PINN and differentiable simulation are complementary . . . 13

9 Game-aware and hybrid extensions 13
9.1 One state, several objectives and costates . . . 13
9.2 Represent both sides of an impulse . . . 13
9.3 A switched rate has a different boundary condition . . . 14

10 Node-level and degree-level inference 14
10.1 Build the network input before the loss . . . 14
10.2 Observation masks define the generalization question . . . 14
10.3 Resolve parameter scaling ambiguities explicitly . . . 15

11 Adapt the implementation through a single sequence 15
11.1 Read the first example before the extensions . . . 15
11.2 Run a focused extension from Python . . . 15
11.3 From inference to feedback decisions . . . 16

12 Use the cyber-control papers to design meaningful inverse problems 16
12.1 State estimation before security evaluation or control . . . 16
12.2 Learning a parameter and learning a missing flux are different tasks . . . 16
12.3 Two coupled layers and event-conditioned data . . . 16
12.4 Strategic objectives survive a neural solver . . . 17

A A complete ODE-fitting reference 17

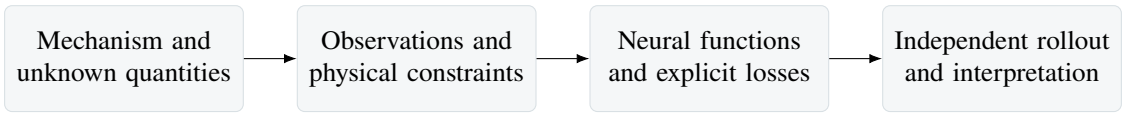
B Short exercises with worked answers **18**

C Development and PDF compilation **18**

1 What is unknown, and what information is available?

Physics-informed learning combines observations with equations that express a proposed mechanism. Its role depends on the unknown object. A forward PINN approximates a trajectory for a known model. An inverse PINN estimates parameters and latent states. A control PINN represents a decision schedule. A PMP-informed method also represents costates and enforces necessary optimality conditions. PIDL is a broader label for learning that uses physical structure, including learned corrections and differentiable simulation [1, 2].

The modeling chain is



The same chain applies to malware, APT and misinformation models. State meanings and intervention effects are defined before selecting the neural architecture.

1.1 Why these inference questions arise in cyber control

The cyber models underlying this tutorial series come from a research line spanning APT security evaluation, patch/repair games and backup/recovery [12, 13, 14, 15], opinion competition and rumor refutation [16, 17], and continuous–impulsive interventions [18, 19, 21]. These models specify physical states, transition mechanisms and decision objectives. In an inverse problem, some of those states or coefficients are unavailable and must instead be inferred from observations.

The awareness-based propaganda and coupled malware–opinion papers make this motivation especially concrete: behavioral states and network coupling influence propagation, while interventions act through different physical channels [22, 23]. The feedback-loop malware-control study explicitly connects parameter learning with MPC [20]. This note develops PINN/PIDL tools that can be used to study related inference or control questions. It does not attribute PINN algorithms to those publications: their original model/control contributions and the neural extensions described here are identified separately.

1.2 Task, output and implementation

Task	Learned object	Repository route
Inverse SIR	$x(t)$ and positive rates	`inverse.py`; the default example.
Missing mechanism	$x(t)$, rates and an additional flux	`pidl.py`.
Direct control	$x(t)$ and bounded $u(t)$	`control.py`.
PMP-informed control	$x(t), u(t), \lambda(t)$	`pmp.py`.
Node-level inference	Node states and selected heterogeneous parameters	`node_problem.py`, `node_inverse.py`.
Feedback HJB/game values	$V(t,x)$ or player-specific $V_i(t,x)$	A mathematical extension, not the default executable experiment.

1.3 Install and run

Inside `note2-pinn-pidl-cyber-control`, use a separate environment:

```
python -m venv .venv
# macOS / Linux:
```

```
source .venv/bin/activate
# Windows PowerShell instead:
# .venv\Scripts\Activate.ps1
python -m pip install -e .
python -m cyberpinn
```

Use Python 3.11 or 3.12 and install Git for the pinned shared-library dependency. The first experiment runs on a CPU and generates its own observations. It compares a PINN trajectory, an ODE rollout using the PINN's learned rates, and a direct ODE least-squares fit to the same data. No GPU, external dataset or LaTeX installation is needed for this experiment.

The default uses 24 observations, 64 collocation points, a two-hidden-layer width-32 state network, 1,500 Adam steps, and seed 1. These are inspectable teaching settings. The wider study has its own profiles and is invoked separately after the first example is understood.

2 Mechanistic models and observation maps

2.1 The SIR model used for parameter learning

The first inverse problem is

$$\dot{S} = -\beta SI, \qquad \dot{I} = \beta SI - \gamma I, \qquad \dot{R} = \gamma I, \tag{1}$$

with known initial state $(0.95, 0.05, 0)$ and unknown positive rates β, γ. Time is measured in a chosen unit; both rates have inverse-time units. Synthetic truth uses $\beta = 0.8$, $\gamma = 0.2$ on $[0, 20]$. The values illustrate the procedure and are not fitted measurements of malware incidents.

The physical state is nonnegative and sums to one. Each infection flux βSI leaves S and enters I; each recovery flux γI leaves I and enters R. The vector field is inward-pointing at each nonnegativity boundary. These are properties of the model to preserve when adding controls or learned mechanisms.

2.2 Measurements are not latent-state truth

Write the observation equation explicitly:

$$y_j = h(x(t_j), \theta) + \varepsilon_j. \tag{2}$$

For infection fractions, $h(x) = I$. For expected counts in a population of size n, $h(x) = nI$. A count observation may need a count likelihood rather than squared error in unscaled proportions. Known initial conditions, known controls, observed compartments and unknown parameters should be stated separately.

The function `observation_data` returns the full time grid, clean synthetic truth, observed indices and measured infection values as separate objects. A local seeded generator adds noise only to the observations. Both the PINN and the ODE baseline receive those same values. Noise does not alter the clean latent states used for evaluation. The additive Gaussian measurement example permits noisy values outside $[0, 1]$; another measurement process can use an appropriate bounded or count model.

For partial multivariate data, let M_{jc} indicate whether component c is observed at time t_j. A masked squared error is

$$\mathcal{L}_{\text{data}} = \frac{\sum_{j,c} M_{jc} [\hat{x}_c(t_j) - y_{jc}]^2}{\sum_{j,c} M_{jc}}. \tag{3}$$

Missing values contribute no term. Replacing them with zeros and treating those zeros as observations changes the inference problem.

2.3 Other cyber mechanisms retain the same workflow

An APT model with vulnerable, compromised, detected and protected states is

$$\begin{aligned} \dot{V} &= -\beta VC - u_p V + \omega P, & \dot{C} &= \beta VC - (\alpha + u_m)C, \\ \dot{D} &= (\alpha + u_m)C - u_r D, & \dot{P} &= u_p V + u_r D - \omega P. \end{aligned} \tag{4}$$

Patching, monitoring and repair operate on different transitions. An inverse task might estimate β and α from partial detection records; a control task might optimize u_p, u_m, u_r with the rates fixed. Both use the same physical state, but different trainable objects and objectives.

A misinformation model can separate unaware users, misinformation spreaders, counter-message spreaders and inactive users. Counter-messaging may change transmission/conversion rates or cause a campaign reset. The observation map then describes what a dataset actually records—posts, accounts, aggregate prevalence or sampled attitudes—rather than assuming that the complete mathematical state is observed.

3 Represent functions and constraints in a neural model

3.1 The state network and time scaling

An MLP approximates a function through compositions of affine maps and nonlinear activations. A trajectory network takes time as input and produces state logits. In the inverse SIR implementation,

$$\hat{x}(t) = \mathrm{softmax}(N_\phi(t/10 - 1)), \qquad t \in [0, 20]. \tag{5}$$

The softmax acts across the three compartments at each time. It enforces sum-to-one and positive outputs for finite logits. The known initial $R(0) = 0$ is therefore approached through an initial-condition loss, rather than attained exactly by a finite softmax logit.

For $\tau = t/T$, the chain rule is $\mathrm{d}\hat{x}/\,\mathrm{d}t = T^{-1}\,\mathrm{d}\hat{x}/\,\mathrm{d}\tau$. For $\tau = 2t/T - 1$, it is

$$\frac{\mathrm{d}\hat{x}}{\mathrm{d}t} = \frac{2}{T}\frac{\mathrm{d}\hat{x}}{\mathrm{d}\tau}. \tag{6}$$

In the supplied inverse network, scaling occurs inside `forward`, while differentiation is requested with respect to physical t. Autograd therefore supplies the scale factor automatically. If the independent differentiable input is already τ, the factor must be supplied explicitly. The two conventions should not be mixed.

3.2 Hard initial conditions and physical state constraints

The ansatz $\hat{x}(t) = x_0 + tN_\phi(t)$ enforces $\hat{x}(0) = x_0$, but imposes neither conservation nor nonnegativity. Applying a softmax afterward changes the initial value in general. A simplex-preserving alternative on $t \geq 0$ is

$$\hat{x}(t) = e^{-ct}x_0 + (1 - e^{-ct})\,\mathrm{softmax}(N_\phi(t)), \qquad c > 0. \tag{7}$$

It enforces the initial state and simplex membership, but also structures the initial derivative and may restrict approximation for a fixed c. The first example uses the simpler soft initial-condition loss, keeping this modeling choice visible.

3.3 Parameter, control and costate outputs

Positive rate parameters can use

$$\hat{\beta} = \mathrm{softplus}(b_\beta) + \epsilon, \qquad \mathrm{softplus}(z) = \log(1 + e^z). \tag{8}$$

This guarantees positivity, not a known upper bound or identifiability. A specified finite interval can instead use a scaled sigmoid. Those bounds are assumptions that should have an application-specific justification.

A control network $N_u(t)$ represents an open-loop schedule. A policy $\pi(t, x)$ represents feedback only if it is trained and assessed over the relevant state domain. A family of time schedules indexed by an initial state is also not automatically a policy reacting to new measurements. Costates can have either sign and generally require an unconstrained output, rather than softmax or sigmoid.

3.4 Differentiate each output component

For a batch of independent time inputs and a pointwise MLP, the derivative array should have the same shape as the predicted states:

Listing 1: Componentwise physical-time derivatives

```
import torch

def time_derivative(values, time):
    columns = []
    for j in range(values.shape[1]):
        derivative = torch.autograd.grad(
            values[:, j].sum(), time, create_graph=True
        )[0]
        columns.append(derivative)
    return torch.cat(columns, dim=1)
```

Using `autograd.grad(values.sum(), time)` differentiates the sum of all components. For simplex states that sum is one, so the result can be zero even when every compartment is changing. The componentwise construction avoids this cancellation. The batch-sum shortcut assumes independent samples; a network that couples different batch rows requires more careful Jacobian handling [3].

`create_graph=True` allows the subsequent residual loss to train the network through these derivatives. Detaching the derivative or the Hamiltonian partials would remove that learning path. Leaf collocation-input gradients can be reset with `t.grad = None` between iterations; resetting model parameter gradients alone does not clear them.

4 Forward and inverse PINN losses

4.1 Write the residual before writing the optimizer

For a known control and parameter vector, define

$$r_x(t) = \dot{\hat{x}}(t) - f(\hat{x}(t), u(t), \theta, t). \tag{9}$$

For inverse SIR, the three residuals are

$$\begin{aligned} r_S &= \dot{\hat{S}} + \hat{\beta}\hat{S}\hat{I}, \\ r_I &= \dot{\hat{I}} - \hat{\beta}\hat{S}\hat{I} + \hat{\gamma}\hat{I}, \\ r_R &= \dot{\hat{R}} - \hat{\gamma}\hat{I}. \end{aligned} \tag{10}$$

Let MSE mean the average of squared entries in its entire argument. Then the actual loss structure is

$$\mathcal{L} = \mathrm{MSE}(\hat{I}(t_j^d) - y_j) + w_0\,\mathrm{MSE}(\hat{x}(0) - x_0) + w_f\,\mathrm{MSE}(r_x(t_j^f)). \tag{11}$$

A mean over both time and state components differs by a dimension factor from a mean over time of squared vector norms. Keep the convention fixed when copying loss weights. The default inverse example uses $w_0 = 10$ and $w_f = 1$.

Observation points t_j^d carry measured information. Collocation points t_j^f enforce the assumed equations without adding measured data. For a forward problem, parameters are fixed and the data term may be absent. For an inverse problem, both network weights and selected physical parameters are optimized.

4.2 One training iteration in the repository

Listing 2: The inverse learning loop in conceptual form

```
optimizer.zero_grad()
t_f.grad = None
beta, gamma = positive(beta_raw), positive(gamma_raw)
loss_data = ((state_net(t_data)[:, 1:2] - I_data) ** 2).mean()
loss_ic = ((state_net(t_zero) - x0) ** 2).mean()
x = state_net(t_f)
r = time_derivative(x, t_f) - sir_rhs(x, beta, gamma)
loss_ode = (r ** 2).mean()
loss = loss_data + w_ic * loss_ic + w_ode * loss_ode
loss.backward()
optimizer.step()
```

The trained parameters are represented by unconstrained raw scalars and transformed within the live graph. The state network uses the same physical RHS as the data-generation model in the matched synthetic experiment. The saved inverse history records rate estimates after the logged optimizer update; its loss values are those used to compute that update.

4.3 Scale losses and sample time deliberately

State fractions, rates, time derivatives and objectives have different units. Nondimensionalization or physically motivated scale factors can make their losses comparable. If observation components have known noise scales, weighting by those scales is often more meaningful than balancing arbitrary raw magnitudes. A high weight encourages a residual to be small at sampled points; it is not an exact equation constraint.

Uniform collocation is easy to follow. Fresh validation times show whether a low training residual extends between those samples. Sharp changes may require denser samples, adaptive placement or interval decomposition. Training can start with a simple forward task, then a small inverse problem, and then control or game residuals. Adam and optional L-BFGS are numerical choices rather than universal guarantees [4, 5]. The default example uses a bounded Adam run rather than an automatically expanding training schedule.

5 The first experiment: fit, replay and compare

5.1 Use an ordinary ODE fit as a reference

When only a few parameters of a known ODE are unknown, direct ODE fitting is a meaningful baseline. The code estimates positive rates through their logarithms:

$$\min_{\eta_\beta, \eta_\gamma} \sum_j \left[I_{\mathrm{ODE}}(t_j; e^{\eta_\beta}, e^{\eta_\gamma}) - y_j \right]^2 . \tag{12}$$

`fit_ode` uses SciPy `least_squares` and `solve_ivp`, the same observed times/values and the same initial state. The teaching rate bounds are $[0.01, 3]$ for both rates. These bounds should be reconsidered for another application, and they differ from the PINN's positive-but-unbounded softplus parameterization.

The ODE solver propagates the specified mechanism. A neural trajectory is an independently optimized function encouraged to satisfy that mechanism. To compare the two meanings, the example also integrates

the ODE using the PINN's estimated $(\hat{\beta}, \hat{\gamma})$. Agreement of that replay with the neural trajectory is informative about physical consistency; their disagreement points to a question about training accuracy, information or model assumptions.

5.2 Inspect the six outputs

The output directory is `results/`. Its two figures are `fit.png` and `learning.png`. The accompanying tables are `observations.csv`, `trajectories.csv`, `comparison.csv` and `training.csv`. The fit plot contains clean truth, measured infection values, the neural trajectory, PINN-rate ODE replay and ODE least squares. The loss plot separates unweighted data, physical and initial-condition terms.

The comparison table reports estimated rates, error against the measured values, and error against clean synthetic infection at unobserved grid times. These held-out times lie *inside* the fitted interval; the physics collocation still spans that interval. The metric tests interpolation, not future forecasting. For a forecasting study, fit observations and collocation only on the declared training interval, and specify how the model is propagated beyond it.

```
python -m cyberpinn --iters 3000 --observations 20 --noise 0.01 --seed 4 --output-
    dir results/noisy
```

Changing the number of steps changes optimization effort. Changing observations, noise or the mechanism changes the inference problem. Use the same observed data for each compared method and distinguish uncertainty across training seeds from uncertainty in the measurements.

5.3 Identifiability is an information question

In early SIR growth with $S(t) \approx S_0$, the infected state approximately satisfies

$$\dot{I} \approx (\beta S_0 - \gamma) I. \tag{13}$$

A short early-time infection curve therefore primarily informs the combination $\beta S_0 - \gamma$. More informative times, known interventions, additional observed components or prior information can help separate rates. A small residual alone does not supply the missing distinction.

Structural identifiability asks whether ideal observations uniquely determine a parameter within the model. Practical identifiability concerns the actual noisy, finite observation set. Multiple initial guesses, data windows and sensitivity analyses investigate different aspects of that question. A stable optimizer outcome may still reflect a strong prior or parameterization. With no informative data and unknown parameters, one can compute scenarios consistent with assumptions; unique empirical recovery requires additional information.

6 Neural optimal control

6.1 Distinguish a decision variable from a physical parameter

After identifying a model, hold its chosen parameters fixed and ask which control minimizes

$$J(u) = \int_0^T L(x, u, t)\, \mathrm{d}t + \Phi(x(T)), \qquad \dot{x} = f(x, u, t), \quad x(0) = x_0. \tag{14}$$

For SIR patching,

$$\dot{S} = -\beta SI - uS, \qquad \dot{I} = \beta SI - \gamma I, \qquad \dot{R} = \gamma I + uS, \tag{15}$$

with $0 \le u \le \bar{u}$, and

$$L = qI + \frac{b}{2}u^2, \qquad \Phi = q_T I(T), \qquad b > 0. \tag{16}$$

This control protects susceptible mass. Recovery control would act on infected mass and would give a different optimality formula. Jointly learning poorly identified parameters and an intervention can confound estimation with decision design; a staged identification–control formulation makes the distinction visible.

6.2 Direct neural control

A state network $\hat{x}(t)$ and a bounded schedule network $\hat{u}(t)$ can be trained using

$$\mathcal{L}_{\text{direct}} = w_J \widehat{J} + w_f \,\text{MSE}(\dot{\hat{x}} - f(\hat{x}, \hat{u})) + w_0 \,\text{MSE}(\hat{x}(0) - x_0). \tag{17}$$

For an equally spaced grid, use an appropriate quadrature for the integral in $\widehat{J}$; for uniformly sampled times a Monte Carlo estimate is T times the sample mean of the running cost. The horizon factor is essential when balancing running and terminal cost. Nonuniform sampling needs corresponding weights.

Finite loss weights permit tradeoffs between objective reduction and imperfect dynamics. Evaluate the learned control by an independent ODE rollout $x_{\hat{u}}$ and compute

$$J_{\text{replay}}(\hat{u}) = \int_0^T L(x_{\hat{u}}(t), \hat{u}(t), t)\,\mathrm{d}t + \Phi(x_{\hat{u}}(T)). \tag{18}$$

This is the physical objective of the proposed schedule. Computing cost only on an inaccurate neural state curve can underestimate the cost of applying that schedule.

6.3 PMP-informed control derives a different loss

Define $H = L + \lambda^\top f$. For the normal minimization problem with a free terminal state,

$$\dot{x} = f, \qquad \dot{\lambda} = -H_x, \qquad \lambda(T) = \Phi_x(x(T)), \qquad u \in \operatorname*{arg\,min}_{v \in [0, \bar{u}]} H(x, v, \lambda, t). \tag{19}$$

These necessary conditions motivate state, costate and control networks. They are distinct from a direct cost-penalty formulation, and do not by themselves prove global optimality [6, 7].

For (15)–(16),

$$\begin{aligned} H &= qI + \frac{b}{2}u^2 + \lambda_S(-\beta SI - uS) + \lambda_I(\beta SI - \gamma I) + \lambda_R(\gamma I + uS), \\ \dot{\lambda}_S &= \beta I(\lambda_S - \lambda_I) + u(\lambda_S - \lambda_R), \\ \dot{\lambda}_I &= -q + \beta S(\lambda_S - \lambda_I) + \gamma(\lambda_I - \lambda_R), \\ \dot{\lambda}_R &= 0, \qquad \lambda(T) = (0, q_T, 0)^\top. \end{aligned} \tag{20}$$

The unconstrained Hamiltonian minimizer is $S(\lambda_S - \lambda_R)/b$. Under the control interval,

$$u_H = \text{clip}\left(\frac{S(\lambda_S - \lambda_R)}{b}, 0, \bar{u}\right). \tag{21}$$

Since $H_{uu} = b > 0$, this is its exact pointwise minimizer.

6.4 The correct residual at a control bound

The first-order condition on the interval is

$$H_u = 0 \text{ if } 0 < u < \bar{u}, \quad H_u \geq 0 \text{ if } u = 0, \quad H_u \leq 0 \text{ if } u = \bar{u}. \tag{22}$$

A single projected residual covers all three cases:

$$r_u = u - \text{Proj}_{[0,\bar{u}]}(u - H_u/b) = u - u_H. \tag{23}$$

For example, $H(u) = u^2/2 + 2u$ is minimized at $u = 0$ on $[0, 1]$, although $H_u(0) = 2$. The projected residual is zero there; a raw H_u^2 penalty is not.

The PMP loss uses

$$r_x = \dot{\hat{x}} - f(\hat{x}, \hat{u}), \qquad r_\lambda = \dot{\hat{\lambda}} + H_x(\hat{x}, \hat{u}, \hat{\lambda}),$$
$$\mathcal{L}_{\text{PMP}} = w_x \,\text{MSE}(r_x) + w_\lambda \,\text{MSE}(r_\lambda) + w_u \,\text{MSE}(r_u) + w_b \mathcal{L}_{\text{boundary}}, \tag{24}$$

where $\mathcal{L}_{\text{boundary}}$ includes the initial state and terminal costate. The `pmp.py` implementation uses `projected_control_residual`; its coefficient `B` is b and its state-cost coefficient `A` is q.

Listing 3: The bounded Hamiltonian condition used in training

```
H = hamiltonian(x, u, lam, A, B, beta, gamma)
Hx = torch.autograd.grad(H.sum(), x, create_graph=True,
                         retain_graph=True)[0]
Hu = torch.autograd.grad(H.sum(), u, create_graph=True,
                         retain_graph=True)[0]
r_u = u - torch.clamp(u - Hu / B, min=0.0, max=umax)
loss_stationarity = (r_u ** 2).mean()
```

The state, costate and control networks are separate functions of time, so these are partial derivatives with respect to their displayed outputs. If a control is analytically substituted as a function of x and λ, derive the required partials before substitution or account carefully for the computation graph. Differentiating a substituted composite Hamiltonian can introduce unintended chain-rule terms.

For a general differentiable Hamiltonian, $u - \text{Proj}_U(u - \eta H_u)$ is a constrained first-order residual for $\eta > 0$. A zero residual is a pointwise minimum when the Hamiltonian is convex in u; otherwise it is only a stationary candidate. For a nonconvex or discrete action set, use the actual admissible Hamiltonian minimization rather than pretending that interval projection solves it.

7 What accuracy and optimality can be concluded?

7.1 Separate the four outputs of an inverse-control study

Data agreement concerns the observation map. Trajectory accuracy concerns the whole state. Parameter recovery concerns the unknown coefficients. Control performance concerns the independently replayed objective. A result can be strong on one of these and weak on another. Choose the comparison that answers the intended scientific question.

For synthetic truth, report state and parameter errors on a separate grid. For real data, true hidden states and rates are usually unavailable; held-out measurements and sensitivity to assumptions then become central. For control, compare replayed state/control curves and cost components under matched constraints and initial states. FBSM, direct transcription, no intervention and informative static/feedback strategies provide different reference points.

7.2 A residual-to-trajectory estimate

The following elementary estimate explains why physical residuals are useful, while also showing the assumptions needed to interpret them.

Proposition 7.1 (Residual consistency for a fixed known model)**.** *Let y solve $\dot{y} = f(y, u, t)$ with initial state y_0. Let $\hat{x}$ be differentiable and $r = \dot{\hat{x}} - f(\hat{x}, u, t)$. Suppose f is Lipschitz in state with constant L on a domain containing both trajectories. Then*

$$\|\hat{x}(t) - y(t)\| \le e^{Lt} \,\|\hat{x}(0) - y_0\| + \int_0^t e^{L(t-s)} \,\|r(s)\| \,\mathrm{d}s. \tag{25}$$

Proof. Subtract the state equations and use the integral form of the error. Lipschitz continuity bounds the vector-field difference by $L\,\|\hat{x} - y\|$. Gronwall's inequality gives (25). □

The estimate uses the residual throughout the interval, not only its finite training-point average. Long horizons, large L, stiff dynamics and missed residual peaks can enlarge error. With uncertain parameters, the difference between $f(\cdot, \hat{\theta})$ and $f(\cdot, \theta^*)$ introduces an additional forcing term; small residual under the learned parameters alone does not bound error against an unknown true mechanism.

For a reset with Lipschitz constant L_G and jump residual $r_G = \hat{x}^+ - G(\hat{x}^-, v)$, the corresponding event error obeys

$$\left\|\hat{x}^+ - y^+\right\| \le L_G\,\|\hat{x}^- - y^-\| + \|r_G\|\,. \tag{26}$$

This connects interval accuracy to event accuracy without forcing a smooth approximation through a jump.

7.3 Necessary conditions and sufficient verification

A small PMP loss is evidence about the sampled necessary conditions. Global sufficiency can follow from appropriate joint Hamiltonian/terminal convexity, or from a value-function verification theorem, under their assumptions. Nonlinear SIR dynamics do not automatically satisfy joint convexity merely because the control cost is quadratic.

For a smooth candidate value function in a minimization problem,

$$V_t + \min_u\{L + \nabla V^\top f\} = 0, \qquad V(T, x) = \Phi(x) \tag{27}$$

provides a different route. Along any admissible trajectory, $L + V_t + \nabla V^\top f \ge 0$; equality for a minimizing feedback selector verifies optimality. Residual-based PDE approaches such as DGM and differential-equation libraries such as DeepXDE provide related computational tools [11, 10]. A neural HJB residual sampled at finitely many state-time points is an approximation to that global statement. The state-domain coverage and approximation error remain part of the assessment.

8 PIDL: learn a structured missing mechanism

8.1 Conservation and positivity are separate properties

The synthetic PIDL example extends SIR with an extra transmission flux:

$$f_{\text{true}}(x) = f_{\text{SIR}}(x; \beta, \gamma) + (-1, 1, 0)^\top q S I^2, \qquad q \ge 0. \tag{28}$$

Opposite signs preserve the sum of compartments. But a generic nonnegative correction $g_\phi(x)$ subtracted from S can still make $\dot{S} < 0$ at $S = 0$. Conservation alone does not preserve nonnegativity.

Use a nonnegative, state-gated flux,

$$g_\phi(x) = SI\,\text{softplus}(N_\phi(x)), \qquad f_{\text{PIDL}} = f_{\text{SIR}} + (-1, 1, 0)^\top g_\phi(x). \tag{29}$$

The extra transmission vanishes at $S = 0$ and $I = 0$, matching the contact-driven mechanism. At $R = 0$, recovery contributes a nonnegative derivative. Thus the physical simplex remains invariant for the continuous model. The correction network returns the *flux* g_ϕ, not merely the unconstrained rate output.

Listing 4: The physical structure is visible in the correction

```
class CorrectionNet(torch.nn.Module):
    def __init__(self, width=64, depth=2):
        super().__init__()
        self.net = MLP(3, 1, width=width, depth=depth)
```

```
    def forward(self, x):
        rate = torch.nn.functional.softplus(self.net(x))
        return x[..., 0:1] * x[..., 1:2] * rate
```

For a different missing mechanism, derive its source compartment, destination compartment and boundary behavior first. External seeding, for example, need not vanish at $I = 0$ and would require a different interpretation. A sign or state gate should express a scientific assumption, not merely improve numerical appearance.

8.2 A correction is not automatically an identified mechanism

The code learns positive base rates as well as a flexible correction. In (29), the total infection flux is $SI[\beta + \mathrm{softplus}(N_\phi(x))]$. A constant contribution can be shifted between the base rate and the neural rate while preserving the total over the observed region. Regularizing the correction selects among explanations; it does not create unique information about their decomposition.

Compare the known-only model with the corrected model on the same observations and held-out trajectories. Examine the total effective flux, parameter sensitivity and behavior under new initial conditions. A small, stable correction can be useful for prediction or control even when its decomposition into a named physical cause is not uniquely identified.

8.3 PINN and differentiable simulation are complementary

A PINN represents the trajectory and penalizes its differential residual. A neural ODE or differentiable-simulation model represents a vector field and obtains trajectories by integration [8]. Both can use a structured correction. Their numerical errors and optimization costs differ: collocation residuals assess local equation agreement, while integration propagates a vector field through time. Independent replay remains useful in either case.

9 Game-aware and hybrid extensions

9.1 One state, several objectives and costates

For a two-player open-loop game with costs $J_i = \Phi_i + \int L_i \, \mathrm{d}t$, define $H_i = L_i + \lambda_i^\top f$. A candidate has one shared state equation and, for each player,

$$\dot{\lambda}_i = -(H_i)_x, \qquad \lambda_i(T) = (\Phi_i)_x, \qquad u_i \in \arg\min_{v_i \in U_i} H_i(x, v_i, u_{-i}, \lambda_i, t). \tag{30}$$

Use the appropriate maximization convention if a player is defined by payoff. For convex action sets, projected residuals can express each player's constrained first-order condition. A scalar sum of residual losses is a numerical device for solving these equations; it is not the sum of players' economic objectives.

A time-only game PINN produces open-loop schedules for the specified initial state. A feedback Nash formulation instead involves player-specific value functions and coupled HJB equations. Neither form is certified merely by a small empirical residual. Frozen-opponent unilateral-response searches provide useful evidence; their attained improvements are lower bounds on global exploitability, unless the best-response problem is solved with a suitable guarantee.

9.2 Represent both sides of an impulse

For an event at τ,

$$\dot{x} = f(x, u, t), \quad t \neq \tau, \qquad x(\tau^+) = G(x(\tau^-), v). \tag{31}$$

Use separate interval networks, or a representation explicitly capable of a discontinuity. A single smooth network evaluated at the same time cannot provide two different one-sided values. Define left and right endpoint predictions and include

$$r_G = \hat{x}^+(\tau) - G(\hat{x}^-(\tau), v). \tag{32}$$

For a continuous–impulsive optimal-control problem, add the costate jump residual

$$r_{\lambda,G} = \hat{\lambda}^- - G_x^\top \hat{\lambda}^+ - K_x. \tag{33}$$

A free interior impulse magnitude satisfies the appropriate first-order condition involving $K_v + G_v^\top \lambda^+$. A freely optimized interior event time additionally has

$$H^- - H^+ + K_t + (\lambda^+)^\top G_t = 0 \tag{34}$$

under the smooth unconstrained-timing assumptions. Guards, active spacing constraints and endpoint events require their own conditions. Fixed event times do not need a free-time residual.

For interval $[\tau_\ell, \tau_{\ell+1}]$, local time $s = (t - \tau_\ell)/h_\ell$, $h_\ell = \tau_{\ell+1} - \tau_\ell$, gives $\dot{x} = h_\ell^{-1} \partial_s x$. If event times are trainable, the duration enters both the residual and objective quadrature. Ordered times and any minimum spacing must be enforced by the optimization formulation.

9.3 A switched rate has a different boundary condition

A sampled action can change the vector field while the state remains continuous. Then interval networks should match state values at the decision boundary, without an impulse reset residual. The time derivative may have a kink. A mode–intensity action is not by itself evidence of a state jump. For actual impulse decisions, the value-function obstacle in a minimization problem is $\mathcal{M}V - V \geq 0$, consistent with the QVI developed in the foundation note.

10 Node-level and degree-level inference

10.1 Build the network input before the loss

For node-level propagation, construct A_{ij} so that node j influences node i, and preserve node ordering in every state, parameter and observation array. A sparse incoming pressure is

$$p_i(t) = \sum_j A_{ij} b_j I_j(t). \tag{35}$$

A heterogeneous node-SIPS model can use $s_i S_i p_i$ as the infection flux, with recovery, protection and intervention efficacies attached to their corresponding transitions. The node state has three compartments per node, whereas the simplest degree SIS state has one infection fraction per degree class. Their tensor dimensions and constraints differ.

For a degree model, calculate $N_k, P(k), \bar{k}$ from the full input graph. The uncorrelated closure uses $\Theta = \sum_k kP(k)x_k/\bar{k}$. A network with correlations can use a conditional mixing matrix. Degree is an unweighted neighbor count unless a weighted closure is specifically derived; in-degree alone does not specify a directed mean-field model.

10.2 Observation masks define the generalization question

The node-level code separates observed nodes, observed times and held-out combinations. Holding out time on observed nodes tests temporal interpolation or forecasting depending on the split. Holding out nodes at observed times tests spatial reconstruction. Holding out both tests a joint task. Report the actual mask used for each metric; averaging all unobserved entries into one number can obscure their distinct difficulties.

A dense time-to-all-nodes network has an output dimension tied to the chosen graph. A factorized network uses time and node features to generate node predictions with shared weights. Such sharing permits a transfer experiment, but useful transfer still depends on feature meaning, graph distribution and the learned mechanism. Architecture comparisons should state capacity, observations and training effort.

10.3 Resolve parameter scaling ambiguities explicitly

If infection contains a product $s_i A_{ij} b_j$, then for $c > 0$ the transformation $s_i \mapsto c s_i$, $b_j \mapsto b_j / c$ leaves that product unchanged. A normalization such as a unit geometric mean for infectivity fixes a representation. It does not add an extra measured quantity or prove that susceptibility and infectivity are individually identifiable from the available data.

This distinction is useful when combining inference with control: predictions may determine an effective transmission operator sufficiently well for a decision even while separate biological/social interpretations remain uncertain. Evaluate the decision using that uncertainty rather than relying on labels alone.

11 Adapt the implementation through a single sequence

11.1 Read the first example before the extensions

Read `cli.run_example`, then `inverse.observation_data`, then `inverse.train`. The first file assembles the same-data comparison and replay. The second defines precisely what is observed. The third expresses the neural approximation and losses. The shared library supplies the MLP, physical RHS and derivative helper rather than duplicating them in every example.

For a new model, align the state ordering, observation map, parameter constraints, physical-time scale, RHS, neural outputs and evaluation solver. When adding control, also derive the objective, Hamiltonian, costate boundary conditions and bounded optimizer. When adding impulses, derive left/right state and costate relations before training. This keeps each loss tied to a mathematical statement.

11.2 Run a focused extension from Python

Listing 5: Train the bounded PMP example through its existing API

```
from cyberpinn.configs import PMPConfig
from cyberpinn.pmp import train

cfg = PMPConfig(
    iters=500, width=32, depth=2, n_collocation=64,
    device="cpu", log_every=100, seed=4
)
state_net, costate_net, control_net, history = train(cfg)
print(history[-1])
```

The call illustrates the interface and an explicitly bounded training budget. Its final control should then be replayed and compared with the same physical model. Increasing the training budget is justified by measured accuracy and decision behavior, rather than by assuming that a longer run must converge.

The existing broader study is available with

```
python -m cyberpinn study --device cpu --output-dir results/study
```

It has separate method profiles. The default first-example flags do not redefine every method in that preset. The ordinary entry point remains `python -m cyberpinn`.

11.3 From inference to feedback decisions

A practical application can observe the system, estimate latent states/rates, choose a control, apply it, and update from new observations. Each stage has different information and uncertainty. A fitted time-only trajectory does not automatically supply an online estimator, and a fitted time-only control does not automatically supply feedback. The foundation note develops continuous and hybrid decision theory; Note 1 develops sampled policy learning. Their interfaces can be combined once the observation and decision meanings agree.

12 Use the cyber-control papers to design meaningful inverse problems

12.1 State estimation before security evaluation or control

Yang et al.'s APT security-evaluation model relates dynamical behavior to a security assessment [12]. A related inverse study must first specify which of those states are observed and which mechanisms are known. Observing a security score is not the same as observing every compartment or node. Define the observation operator before choosing a network output or residual.

The patch-injection, repair and backup/recovery studies then motivate decision-relevant inference [13, 14, 15]. For example, uncertainty in recovery capability may matter differently from uncertainty in transmission. The inverse objective should recover the quantities needed by the stated controller, and the final evaluation should replay that controller under a separately specified physical model. Accurate interpolation of observed infection alone does not establish accurate recovery costs or control decisions.

12.2 Learning a parameter and learning a missing flux are different tasks

Jafar et al. use learning to estimate unknown parameters in an MPC feedback loop [20]. This motivates a concrete interface for the PINN example: infer parameters from observations, replay the inferred model, compute an admissible intervention and evaluate the resulting state/cost. Substituting a PINN for the paper's estimator is a new experiment, not an existing result of that paper.

The structured PIDL correction in this note answers a different question: whether a specified mechanistic family omits an additional state-dependent flux. In an awareness-coupled model, an apparent extra infection flux could reflect an unobserved behavioral state rather than a new transmission law [22, 23]. Compare the explicit coupled-state model with an effective correction only after stating what is measured and which parameterizations are distinguishable from those measurements.

12.3 Two coupled layers and event-conditioned data

The malware–opinion paper has coupled infection and awareness dynamics and separate patching/training impulse controls [23]. For a proposed inverse extension, let $c(t)$ and $o(t)$ denote those two state vectors and use an observation map selecting the measured nodes, times and components. The actual physical RHS and reset maps must come from the chosen model. The earlier one-state SIR residual cannot represent that coupling merely by adding more output columns.

Data at an intervention time require a left/right convention. A measurement just before patching is an observation of $c(\tau^-)$; one just after is of $c(\tau^+)$. Awareness training can change o while c remains continuous at that same event. Separate interval networks and a reset residual represent these distinctions. This is a suggested PINN formulation for the model class; the cited publication develops its own impulse-control approach.

The awareness-based propaganda study and its no-awareness comparison also motivate an identifiable ablation [22]: determine whether additional awareness observations improve held-out propagation prediction, rather than using hidden synthetic states during training without declaring them. A model without awareness and a model with unobserved awareness have different statistical meanings.

12.4 Strategic objectives survive a neural solver

The conflicting-opinion and rumor-refutation papers provide examples of player-specific objectives [16, 17]; the propaganda-war paper adds hybrid decisions [21]. A neural solver should retain one costate/value representation per player where the formulation requires it. Adding residuals to one training loss does not turn a general-sum game into a common-payoff problem.

Impulsive artificial defense and outsourced defense extend this lesson to events and decision ownership [18, 19]. A learning-based numerical extension must retain the original continuous/event decision sets, costate jumps and timing constraints. Compare it with the original method on the same mathematical problem before interpreting a difference as improved optimization. Frozen-opponent response tests, independent ODE/event replay and data holdouts answer complementary questions; none replaces the others.

A connected research workflow

Choose a published mechanism and its decision question; state the available measurements; identify the unknown quantities; derive the residuals and constraints; compare inference with a matched mechanistic baseline; and evaluate decisions using independent physical replay. This connects the author's model-based research with new learning tools while preserving the meaning of each contribution.

A A complete ODE-fitting reference

The following compact example shows the baseline idea using standard SciPy functions. It uses observed infection values only; it does not feed hidden state truth into the fit.

Listing 6: Mechanistic fitting with the same observation operator

```
import numpy as np
from scipy.integrate import solve_ivp
from scipy.optimize import least_squares

def trajectory(time, beta, gamma):
    def rhs(t, x):
        S, I, R = x
        infection, recovery = beta * S * I, gamma * I
        return [-infection, infection - recovery, recovery]
    sol = solve_ivp(rhs, (0.0, float(time[-1])),
                    [0.95, 0.05, 0.0], t_eval=time,
                    rtol=1e-8, atol=1e-10)
    if not sol.success:
        raise RuntimeError(sol.message)
    return sol.y.T

# Replace these synthetic observations by a validated data loader.
time = np.linspace(0.0, 20.0, 24)
observed_I = trajectory(time, 0.8, 0.2)[:, 1]
fit = least_squares(
    lambda z: trajectory(time, *np.exp(z))[:, 1] - observed_I,
    np.log([0.7, 0.3]),
    bounds=(np.log([0.01, 0.01]), np.log([3.0, 3.0])))
if not fit.success:
    raise RuntimeError(fit.message)
print("beta, gamma:", np.exp(fit.x))
```

For a general dataset, check increasing unique observation times, physical units, the known/estimated initial state and the measurement model. The library solver's tolerances concern numerical integration; the statistical uncertainty of fitted rates is a separate issue [9].

B Short exercises with worked answers

Time scaling. A network accepts $\tau = 2t/T - 1$. What is the physical derivative if differentiation is taken with respect to τ?

Answer: $(2/T)\partial_\tau \hat{x}$. If scaling is inside a network whose differentiable input remains t, the chain rule is already included.

Summed derivatives. A simplex network has $S + I + R = 1$. Why can differentiating the sum return zero?

Answer: It differentiates the conservation identity. The ODE requires three separate derivatives, which can be nonzero while summing to zero.

Boundary flux. Does $(-g, +g, 0)$ with $g > 0$ preserve the physical simplex at $S = 0$?

Answer: It preserves total mass but points out of the nonnegative region because $\dot{S} = -g < 0$. A contact-driven flux $SIh(x)$ with $h \geq 0$ vanishes on the appropriate boundaries.

Parameter replay. A neural curve fits $I(t)$ but its learned rates generate a different ODE curve. Which output is the physical prediction of those rates?

Answer: The ODE replay. The neural fit is a separate approximation whose residual/initial consistency must be assessed before equating the two.

Optimality at the upper bound. For a minimization Hamiltonian with $u = \bar{u}$ and $H_u < 0$, should the raw derivative be driven to zero?

Answer: No. The derivative points toward increasing u, which is infeasible. The projected residual is zero at the constrained optimum.

C Development and PDF compilation

```
python -m pip install -e ".[dev]"
pytest -q
python -m cyberpinn check
python -m cyberpinn docs
```

Only `docs` requires LaTeX/`latexmk`. The mathematical tests and small execution checks support implementation correctness; parameter recovery, control quality and equilibrium claims require their corresponding data and comparison evidence.

References

[1] M. Raissi, P. Perdikaris and G. E. Karniadakis, "Physics-informed neural networks: A deep learning framework for solving forward and inverse problems involving nonlinear partial differential equations," *Journal of Computational Physics*, 378, 686–707, 2019.

[2] G. E. Karniadakis et al., "Physics-informed machine learning," *Nature Reviews Physics*, 3, 422–440, 2021.

[3] PyTorch developers, *torch.autograd.grad*, official documentation. https://docs.pytorch.org/docs/stable/generated/torch.autograd.grad.html.

[4] S. Wang, Y. Teng and P. Perdikaris, "Understanding and mitigating gradient flow pathologies in physics-informed neural networks," *SIAM Journal on Scientific Computing*, 43(5), A3055–A3081, 2021.

[5] A. Krishnapriyan et al., "Characterizing possible failure modes in physics-informed neural networks," *Advances in Neural Information Processing Systems*, 2021.

[6] S. Lenhart and J. T. Workman, *Optimal Control Applied to Biological Models*. Chapman and Hall/CRC, 2007.

[7] D. Liberzon, *Calculus of Variations and Optimal Control Theory: A Concise Introduction*. Princeton University Press, 2012.

[8] R. T. Q. Chen, Y. Rubanova, J. Bettencourt and D. Duvenaud, "Neural ordinary differential equations," *Advances in Neural Information Processing Systems*, 2018.

[9] SciPy developers, *solve_ivp* and *least_squares*, official documentation. https://docs.scipy.org/doc/scipy/reference/integrate.html; https://docs.scipy.org/doc/scipy/reference/optimize.html.

[10] L. Lu, X. Meng, Z. Mao and G. E. Karniadakis, "DeepXDE: A deep learning library for solving differential equations," *SIAM Review*, 63(1), 208–228, 2021.

[11] J. Sirignano and K. Spiliopoulos, "DGM: A deep learning algorithm for solving partial differential equations," *Journal of Computational Physics*, 375, 1339–1364, 2018.

[12] L.-X. Yang, P. Li, X. Yang and Y. Y. Tang, "Security evaluation of the cyber networks under advanced persistent threats," *IEEE Access*, 5, 20111–20123, 2017. doi:10.1109/ACCESS.2017.2757944.

[13] L.-X. Yang, P. Li, X. Yang, Y. Xiang and W. Zhou, "A differential game approach to patch injection," *IEEE Access*, 6, 58924–58938, 2018. doi:10.1109/ACCESS.2018.2874446.

[14] L.-X. Yang, P. Li, Y. Zhang, X. Yang, Y. Xiang and W. Zhou, "Effective repair strategy against advanced persistent threat: A differential game approach," *IEEE Transactions on Information Forensics and Security*, 14(7), 1713–1728, 2019. doi:10.1109/TIFS.2018.2885251.

[15] L.-X. Yang, K. Huang, X. Yang, Y. Zhang, Y. Xiang and Y. Y. Tang, "Defense against advanced persistent threat through data backup and recovery," *IEEE Transactions on Network Science and Engineering*, 8(3), 2001–2013, 2021. doi:10.1109/TNSE.2020.3040247.

[16] L.-X. Yang, P. Li, X. Yang, Y. Xiang and Y. Y. Tang, "Simultaneous benefit maximization of conflicting opinions: Modeling and analysis," *IEEE Systems Journal*, 14(2), 1623–1634, 2020. doi:10.1109/JSYST.2020.2964004.

[17] D.-W. Huang, L.-X. Yang, P. Li, X. Yang and Y. Y. Tang, "Developing cost-effective rumor-refuting strategy through game-theoretic approach," *IEEE Systems Journal*, 15(4), 5034–5045, 2021. doi:10.1109/JSYST.2020.3020078.

[18] H. Sun, X. Yang, L.-X. Yang, K. Huang and G. Li, "Impulsive artificial defense against advanced persistent threat," *IEEE Transactions on Information Forensics and Security*, 18, 3506–3516, 2023. doi:10.1109/TIFS.2023.3284564.

[19] Y. Qin, X. Yang, L.-X. Yang and K. Huang, "Modeling and study of defense outsourcing against advanced persistent threat through impulsive differential game approach," *Computers & Security*, 145, 104003, 2024. doi:10.1016/j.cose.2024.104003.

[20] M. T. Jafar, L.-X. Yang, G. Li, Q. Zhu and C. Gan, "Minimizing malware propagation in Internet of Things networks: An optimal control using feedback loop approach," *IEEE Transactions on Information Forensics and Security*, 19, 9682–9697, 2024. doi:10.1109/TIFS.2024.3463965.

[21] X. Cheng, L.-X. Yang, Q. Zhu, C. Gan, X. Yang and G. Li, "Cost-effective hybrid control strategies for dynamical propaganda war game," *IEEE Transactions on Information Forensics and Security*, 19, 9789–9802, 2024. doi:10.1109/TIFS.2024.3468903. Author code: PropagandaWar_TIFS_2024_Code.

[22] X. Cheng, L.-X. Yang, Q. Zhu, C. Gan and G. Li, "Impulse strategies for suppressing cyber propaganda with awareness," *IEEE Transactions on Computational Social Systems*, 12(5), 2685–2695, 2025. doi:10.1109/TCSS.2024.3522889. Author code: Propaganda_TCSS_2025_Code.

[23] X. Cheng, L.-X. Yang, G. Li, Z. Ma, T. Zhu, L. Wang and S. Duan, "Modeling and mitigating social engineering malware: Integrating malware-opinion dynamics with optimal impulse control approaches," *IEEE Transactions on Information Forensics and Security*, 20, 12231–12244, 2025. doi:10.1109/TIFS.2025.3630005. Author code: OpinionMalware_TIFS_2025_Code.